An internal solution for calculating proximity-based metrics when using PPI

Presenter:

Steven Lawrence' MS

PhD Student, NYU Grossman School of Medicine

NYU Grossman School of Medicine

Overview

- Background
- Objective
- What is OSRM?
- Proximity calculating Protocol
- Challenges
- Future directions

NYU Grossman School of Medicine

Background

- Chronic health conditions account for 90% of health care cost in the US
- Effective management of chronic disease has been shown to bring down costs
- Better management has been linked to having access to pharmacy by improving one aspect of management, medication adherence

National Center for Disease Chronic Disease Prevention and health Promotion. Chronic disease facts and statistics. https://www.cdc.gov/chronic-14disease/data-research/facts-stats/?CDCAArefV al = https ://www.cdc.gov/chronicdisease/about/costs/index.htm, 2024. Accessed : 2024 – 08 – 28.

Osayi E. Akinbosoye, Michael S. Taitel, James Grana, Jerrold Hill, and Rolin L. Wade. Improving medication adherence and health care outcomes in a commercial population through a community pharmacy. Population Health Management, 19:454–461, 12 2016.

Philippe Amstislavski, Ariel Matthews, Sarah Sheffield, Andrew R. Maroko, and Jeremy Weedon. Medication deserts: survey of neighborhood disparities in availability of prescription medications. International Journal of Health Geographics, 11, 2012.

Background Continued

- Access is most popularly defined as accessibility (proximity) and availability (density)
- Examples of proximity measures are travel time or distance between locations
- An example of availability is the ratio of supply to demand

Brígida Dias Fernandes, Aline Aparecida Foppa, Paulo Henrique, Ribeiro Fernandes Almeida, Ali Lakhani, Tácio De, and Mendonça Lima. Application and utility of geographic information systems in pharmacy specific health research: A scoping review. Research in Social and Administrative Pharmacy, 18:3263–3271, 2022

NYU Grossman School of Medicine

Objective

- Calculate proximity metrics between patient addresses and pharmacies within NYU's firewall
- Use an open source software such as the open street routing machine (OSRM) for its compatibility with R
- Utilize NYU's high performance computing (HPC) resource given the memory requirements of calculating routes through a road network
- Remain HIPPA compliant when using patient protected information (PPI) such as a patient's address

Problem

Current methods for calculating proximity through a road network require cloud usage or proprietary software

- Using cloud service outside of the institutions firewall violates HIPPA regulations and using proprietary software can be limiting

Alternative methods calculate proximity metrics that use PPI by

- Scattering coordinates
- Use a synthetic dataset of the region
- Use centroid as proxy for patient addresses

Kahn, R. M., Ma, X., Gordhandas, S., Yeoshoua, E., Ellis, R. J., Zhang, X., ... & Chi, D. S. (2024). Regionalizing ovarian cancer cytoreduction to high-volume centers and the impact on patient travel in New York State. *Gynecologic Oncology*, *182*, 141-147.

Berenbrok, L. A., Tang, S., Gabriel, N., Guo, J., Sharareh, N., Patel, N., ... & Hernandez, I. (2022). Access to community pharmacies: A nationwide geographic information systems cross-sectional analysis. *Journal of the American Pharmacists Association*, *62*(6), 1816-1822.

Open Street Routing Machine

- An open street routing machine (OSRM) calculates proximity between locations using nodes in an open street map
- Nodes contain information street attributes such as
  - legal speed,
  - what types of turns are allowed,
  - is this street a one way etc.
- Project OSRM can be accessed via the cloud service platform (link) to calculate proximity metrics for driving, foot, and bicycle profiles
  - Route
  - Table
  - Nearest
  - Trip

NYU Grossman School of Medicine

OSRM service and R package

- Project OSRM allows backend and frontend image to be used locally
- Open Street Map extracts can be downloaded (here)
- NYU uses singularity containers which required building the osrm backend from scratch
- Map file is extracted, partitioned, and customized to work with the routing machine housed in a container

Start routing engine

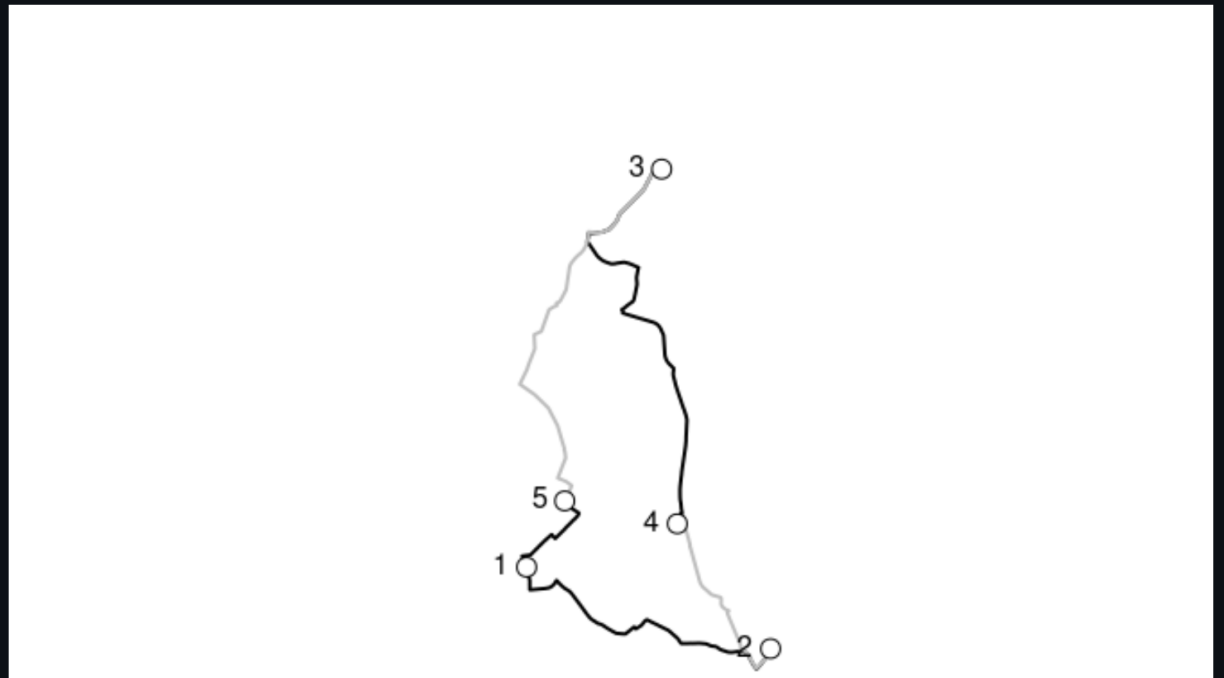
Make http requests

Store routes in a dataframe

OSRM R package (there is support)

- Riatelab has developed R functions that utilize the osrm service (link)
- When utilizing the functions, the local container hosting the routing machine is accessed using http requests
- Customize functions are also able to be created to utilize the osrm service very quickly

```
mytrip <- trips[[1]]$trip
# Display the trip
plot(st_geometry(mytrip), col = c("black", "grey"), lwd = 2)
plot(st_geometry(pharmacy[1:5, ]), cex = 1.5, pch = 21, add = TRUE)
text(st_coordinates(pharmacy[1:5,]), labels = row.names(pharmacy[1:5,]),
     pos = 2)
```



Proximity Calculation Protocol when using HPC

- Load data and code into working directory
  - Using secure copy protocol (SCP)
- Customize job specifications
  - Resources needed
  - Modules used (R, singularity)
- Determines arguments to pass from slurm environment to R code
  - File paths and coordinate column names
  - Ports for singularity container
  - Destinations of where output should be stored

Set up job using slurm code

Allow the R code to wait until the route engine is running

Store routes in a dataframe and extract the proximity metrics

Slurm Code

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=1
#SBATCH --time=08:00:00
#SBATCH --mem=100GB
#SBATCH --job-name=myTest
#SBATCH --output=log_%j.out

module unload gcc
module load r/4.0.3
module load singularity

NODE_IP=$(hostname -I | awk '{print $1}')
DIR='/gpfs/data/adhikarilab/osrm/'
data1= Patient_locations.csv
data2= pharmacy_locations.csv
start_lat=latitude
start_lng=longitude
end_lat=location.lat
end_lng=location.lng
outputname= Patient_to_pharmacy_proximity.csv
port=5000
profile=driving

Rscript get_prox_server_2data.R $NODE_IP $DIR $data1 $start_lat $start_lng $end_lat $end_lng $profile $outputname $port $data2  &

singularity exec --bind "${PWD}/e_car:/e_car" osrm.sif osrm-routed --algorithm mld /e_car/us-latest.osrm
```

Job specifications

NYU Grossman School of Medicine

Slurm Code

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=1
#SBATCH --time=08:00:00
#SBATCH --mem=100GB
#SBATCH --job-name=myTest
#SBATCH --output=log_%j.out

module unload gcc
module load r/4.0.3
module load singularity

NODE_IP=$(hostname -I | awk '{print $1}')
DIR='/gpfs/data/adhikarilab/osrm/'
data1= Patient_locations.csv
data2= pharmacy_locations.csv
start_lat=latitude
start_lng=longitude
end_lat=location.lat
end_lng=location.lng
outputname= Patient_to_pharmacy_proximity.csv
port=5000
profile=driving

Rscript get_prox_server_2data.R $NODE_IP $DIR $data1 $start_lat $start_lng $end_lat $end_lng $profile $outputname $port $data2  &

singularity exec --bind "${PWD}/e_car:/e_car" osrm.sif osrm-routed --algorithm mld /e_car/us-latest.osrm
```

Arguments to pass on

NYU Grossman School of Medicine

Slurm Code

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=1
#SBATCH --time=08:00:00
#SBATCH --mem=100GB
#SBATCH --job-name=myTest
#SBATCH --output=log_%j.out

module unload gcc
module load r/4.0.3
module load singularity

NODE_IP=$(hostname -I | awk '{print $1}')
DIR='/gpfs/data/adhikarilab/osrm/'
data1= Patient_locations.csv
data2= pharmacy_locations.csv
start_lat=latitude
start_lng=longitude
end_lat=location.lat
end_lng=location.lng
outputname= Patient_to_pharmacy_proximity.csv
port=5000
profile=driving

Rscript get_prox_server_2data.R $NODE_IP $DIR $data1 $start_lat $start_lng $end_lat $end_lng $profile $outputname $port $data2  &

singularity exec --bind "${PWD}/e_car:/e_car" osrm.sif osrm-routed --algorithm mld /e_car/us-latest.osrm
```

R code and singularity container specifications

NYU Grossman School of Medicine

R code

```
args = commandArgs(trailingOnly=TRUE) # allows R to accept trailing arguments
# Argument -------------------------
ip = args[1]
DIR = args[2]
data_file_path1 = args[3]
start_lat = args[4]
start_lng = args[5]
end_lat = args[6]
end_lng = args[7]
profile = args[8]
output = args[9]
port = args[10]
data_file_path2 = args[11]

# Reading in data and pkgs---------------------

Packages <- c("tidyverse","httr")

invisible(lapply(Packages, library, character.only = TRUE))
```

Arguments to passed on

NYU Grossman School of Medicine

R code – user defined functions

```
# Defining functions ---------------------
sng_route <- function(i,ip = ip,
            meters_to_miles = 1.60934*1000, # converts meters to miles
            sec_to_min = 60, #converst seconds to minutes
            lat1,
            lng1,
            lat2,
            lng2,
            profile,
            port){



          url = paste0("http://",ip,":",port,"/route/v1/",profile,"/",
                lng1[i], ",", lat1[i], ";",
                lng2[i], ",", lat2[i],
                "?overview=false&steps=true")

          response = GET(url)

          if (status_code(response) == 200) {
            route = content(response, "parsed")
            distance = route$routes[[1]]$distance/meters_to_miles
            duration = route$routes[[1]]$duration/sec_to_min
          } else {
            distance = "error: check data"
            duration = "error: check data"
          }

list(distance, duration)

}
```

NYU Grossman School of Medicine

R code – user defined functions

```
proximity <- function(data, ip = ip,
                      meters_to_miles = 1.60934*1000, # converts meters to miles
                      sec_to_min = 60,
                      lat1,
                      lng1,
                      lat2,
                      lng2,
                      profile,
                      port){

  n_loop <- nrow(data)
  route_out <- vector(length = n_loop)

for (i in 1:n_loop) {

    try(
      route_out[i] <-  sng_route(i =i ,ip = ip,
                     meters_to_miles = meters_to_miles,
                     sec_to_min = sec_to_min,
                     lat1 = lat1,
                     lng1 = lng1,
                     lat2 = lat2,
                     lng2 = lng2,
                     profile = profile,
                     port = port))

}
  data %>% mutate(distance = map_dbl(route_out,1), duration = map_dbl(route_out,2))


}

# merging datasets --------------------
data1 = vroom::vroom(paste0(data_file_path1))
data2 = vroom::vroom(paste0(data_file_path2))

data = merge(data1,data2,by = NULL)


lng1 = data %>% pull(!!sym(start_lng))
lat1 = data %>% pull(!!sym(start_lat))
lng2 = data %>% pull(!!sym(end_lng))
lat2 = data %>% pull(!!sym(end_lat))
```

NYU Grossman School of Medicine

R Code – user defined functions

```
# Check server ----------------


   # Define the coordinates of the start and end points
   start_point <- c(lon = -73.9877376096568, lat = 40.74975622750664)
   end_point <- c(lon = -73.93992875654641, lat = 40.70528988083516)

   # Construct the request URL
   url <- paste0("http://",ip,":",port,"/route/v1/",profile,"/",
                 start_point["lon"], ",", start_point["lat"], ";",
                 end_point["lon"], ",", end_point["lat"],
                 "?overview=false&steps=true")

   server_run_check <- RCurl::url.exists(url)

   while(server_run_check == F){
     Sys.sleep(30)
     server_run_check <- RCurl::url.exists(url)
   }

   # Send the GET request to the OSRM server
   response <- GET(url)

   if (status_code(response) == 200) {
     route = content(response, "parsed")
           route$routes[[1]]$distance
   } else {
     print("error: check data")
   }
```

NYU Grossman School of Medicine

R Code – user defined functions

```
# Check server ----------------


    # Define the coordinates of the start and end points
    start_point <- c(lon = -73.9877376096568, lat = 40.7487522750664)
```

```
# Calculate distances -----------------

if (status_code(response) == 200) {

  data = proximity(data, ip, lat1 = lat1, lng1 = lng1, lat2 = lat2, lng2 = lng2, profile = profile, port = port)

  write.csv(data, paste0(DIR,output,".csv"))
} else {
  print("error: check script or data")
}
```

```
    # Send the GET request to the OSRM server
    response <- GET(url)

    if (status_code(response) == 200) {
      route = content(response, "parsed")
            route$routes[[1]]$distance
    } else {
      print("error: check data")
    }
```

NYU Grossman School of Medicine

Computation challenges

- Calculating proximity between two sets of locations will result in n by m number of calculations
- Current OSRM threading only allows for 4 route calculations at a single time
  - more compute will not speed up the process

NYU Grossman School of Medicine

Future directions

- Test singularity container built to run osrm r package to use isochrone/isodistance functions to reduce calculation time
- Write slurm code that will spin up multiple singularity containers to utilize more compute capacity in a single job (link)
- Make a copy of the OSRM singularity container widely available included the extracted map files
- Add an Open Trip Planner container to work flow

Acknowledgements

- Samrachana Adhikari
- Saul B Blecker
- Amrita Mukhyopadhyay
- Rumi Chunara
- Cassidy Fitchett
- Andrew Fair
- Kevin Yie
- Veronika Bailey
- Tyrel Stokes

NYU Grossman School of Medicine

Thank you!

- Questions?
- Link to example slurm code and R functions

https://github.com/SL-itw/osrm-slurm